



P01-06 · T13–T15 Tauri Shell, Typed IPC v1, First Run & Honest Progress



The rule for this page: **the UI may never lie**. No spinner without a number, no percentage that is a guess, no "Done" until the transaction committed, no progress bar that goes backwards. A photographer who cannot trust the progress bar cannot trust the catalog.

1. `crates/aura-app/src/ipc/contract.rs` — FROZEN (Rust side)

```
///! FROZEN CONTRACT. The IPC boundary between the Rust core and the React UI.  
///! Mirrored byte-for-byte in `ui/src/ipc/types.ts`. Both files are digested  
///! into contracts.lock, and CI fails if one changes without the other.
```

```
use serde::{Deserialize, Serialize};
```

```
/// Every command returns this. There is no other shape.  
///  
/// Why not use Result and let Tauri serialise the error? Because Tauri's default  
/// error path stringifies the error, which loses the code, the severity and the
```

```

/// runbook link. A UI that receives a string can only show a
string. A UI that
/// receives an IpcError can offer the right button.
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(tag = "status", rename_all = "snake_case")]
pub enum IpcResult<T> {
    Ok { data: T },
    Err { error: IpcError },
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct IpcError {
    /// e.g. "AURA-IO-1003". Always present, always renderabl
    e.
    pub code: String,
    /// Plain language, no jargon, no paths. Safe to show in
    a dialog.
    pub user_message: String,
    /// "warning" | "item_failed" | "run_blocking" | "fatal"
    pub severity: String,
    /// "retry" | "fallback" | "quarantine" | "halt" | "ask_u
    ser"
    pub recovery: String,
    /// Deep link such as "aura.app/e/AURA-IO-1003".
    pub runbook_url: String,
    /// True when a Retry button should be offered.
    pub retryable: bool,
}

/// Progress. Sent on the `aura://progress` channel at most 1
0 times a second.
///
/// `total` is Option deliberately. During a scan we genuinel
y do not know how
/// many files exist, and inventing an estimate so the bar ca

```

```

n move is exactly
/// the lie this contract exists to prevent. When total is No
ne the UI shows an
/// indeterminate bar WITH a live count, never a fake percent
age.
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct Progress {
    pub run_id: String,
    /// "scanning" | "hashing" | "indexing" | "finalising"
    pub phase: String,
    pub done: u64,
    pub total: Option<u64>,
    /// Bytes processed so far, for a throughput readout that
is real.
    pub bytes_done: u64,
    /// Only ever Some when at least 5 % is complete AND the
rate over the last
    /// 10 seconds is within 2x of the rate over the whole ru
n. Otherwise None,
    /// and the UI shows no estimate at all.
    pub eta_seconds: Option<u64>,
    /// The current item, already redacted to a file name. Ne
ver a full path:
    /// progress events are logged, and logs must not contain
client paths.
    pub current_item: Option<String>,
    pub quarantined: u64,
    pub cancellable: bool,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct ProjectSummary {
    pub project_id: String,
    pub name: String,

```

```

    pub event_date: Option<String>,
    pub photo_count: u64,
    pub file_count: u64,
    pub bytes_total: u64,
    pub status: String,
    pub consent: ConsentView,
}

/// Consent as the UI sees it. Every field defaults to false;
/// there is no
/// constructor that produces an allowing value without an ex
/// plicit user action.
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct ConsentView {
    pub cloud_metadata: bool,
    pub cloud_derived_imagery: bool,
    pub cloud_full_imagery: bool,
    pub telemetry: bool,
    pub improve_models: bool,
    pub granted_at: Option<String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct ImportRequest {
    pub project_id: String,
    pub roots: Vec<String>,
    /// "reference" | "copy_verified"
    pub mode: String,
    pub extensions: Vec<String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct ImportSummary {

```

```

    pub import_id: String,
    pub files_discovered: u64,
    pub files_imported: u64,
    pub files_already_present: u64,
    pub files_quarantined: u64,
    pub photos_created: u64,
    pub duration_ms: u64,
    /// (code, count, plain-language explanation)
    pub quarantine_groups: Vec<QuarantineGroup>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct QuarantineGroup {
    pub code: String,
    pub count: u64,
    pub explanation: String,
}

/// One grid cell. Deliberately tiny: 4,000 of these must cross the IPC
/// boundary in under 100 ms, so there is no room for a path, a title or an
/// EXIF blob. The UI asks for details only for the cell it is showing large.
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct GridItem {
    pub photo_id: String,
    /// Tier-1 thumbnail path relative to the cache root, or None if not built.
    pub thumb: Option<String>,
    pub capture_time: Option<String>,
    pub orientation: u8,
    pub width: Option<u32>,
    pub height: Option<u32>,
}

```

```

}

/// The IPC surface of phase 01. Fourteen commands, no more.
/// Each is listed here
/// so the contract digest changes if any signature changes.
pub const COMMANDS_V1: &[&str] = &[
    "app_info",
    "first_run_state",
    "choose_catalog_location",
    "create_catalog",
    "open_catalog",
    "close_catalog",
    "create_project",
    "list_projects",
    "get_project",
    "set_consent",
    "start_import",
    "cancel_run",
    "list_grid",
    "list_quarantine",
];

```

2. `ui/src/ipc/types.ts` — FROZEN (TypeScript mirror)

```

// FROZEN CONTRACT. Mirror of crates/aura-app/src/ipc/contract.rs.
// Generated by `cargo xtask ipc-types`; do not hand-edit. CI
// regenerates and
// diffs this file, so an edit on one side without the other
// fails the build.

export type IpResult<T> =
  | { status: "ok"; data: T }

```

```

    | { status: "err"; error: IpcError }

export interface IpcError {
    code: string
    userMessage: string
    severity: "warning" | "item_failed" | "run_blocking" | "fatal"
    recovery: "retry" | "fallback" | "quarantine" | "halt" | "ask_user"
    runbookUrl: string
    retryable: boolean
}

export interface Progress {
    runId: string
    phase: "scanning" | "hashing" | "indexing" | "finalising"
    done: number
    /** null means genuinely unknown. Render an indeterminate bar plus a count. */
    total: number | null
    bytesDone: number
    /** null means we refuse to guess. Render nothing, not "calculating..." */
    etaSeconds: number | null
    currentItem: string | null
    quarantined: number
    cancellable: boolean
}

export interface ConsentView {
    cloudMetadata: boolean
    cloudDerivedImagery: boolean
    cloudFullImagery: boolean
    telemetry: boolean
    improveModels: boolean
    grantedAt: string | null
}

```

```

}

export interface ProjectSummary {
  projectId: string
  name: string
  eventDate: string | null
  photoCount: number
  fileCount: number
  bytesTotal: number
  status: "active" | "archived" | "delivered"
  consent: ConsentView
}

export interface ImportRequest {
  projectId: string
  roots: string[]
  mode: "reference" | "copy_verified"
  extensions: string[]
}

export interface QuarantineGroup {
  code: string
  count: number
  explanation: string
}

export interface ImportSummary {
  importId: string
  filesDiscovered: number
  filesImported: number
  filesAlreadyPresent: number
  filesQuarantined: number
  photosCreated: number
  durationMs: number
  quarantineGroups: QuarantineGroup[]
}

```

```
export interface GridItem {
  photoId: string
  thumb: string | null
  captureTime: string | null
  orientation: number
  width: number | null
  height: number | null
}
```

The typed invoke wrapper — `ui/src/ipc/client.ts`

```
import { invoke } from "@tauri-apps/api/core"
import type { IpcError, IpcResult } from "../types"

/** Thrown for every failed command. Carries the code, so callers can branch. */
export class AuraIpcError extends Error {
  constructor(public readonly detail: IpcError) {
    super(detail.userMessage)
    this.name = "AuraIpcError"
  }
}

/**
 * The only way the UI talks to Rust. Centralising this means:
 * 1. every error becomes an AuraIpcError with a code, never a bare string
 * 2. every call is logged once with its duration
 * 3. there is exactly one place to add a timeout or a retry
 */
export async function call<TArgs extends object, TData>(
  command: string,
```

```

    args?: TArgs,
  ): Promise<TData> {
    const started = performance.now()
    let result: IpcResult<TData>
    try {
      result = await invoke<IpcResult<TData>>>(command, args ??
    {}))
    } catch (cause) {
      // A thrown invoke means the command panicked or does not
      // exist: a bug, not
      // a user error. Surface it as fatal rather than pretendi
      // ng it is handled.
      throw new AuraIpError({
        code: "AURA-IPC-9001",
        userMessage: "AURA hit an internal problem. Please rest
art the app.",
        severity: "fatal",
        recovery: "halt",
        runbookUrl: "aura.app/e/AURA-IPC-9001",
        retryable: false,
      })
    } finally {
      console.debug(`ipc ${command} ${Math.round(performance.no
w() - started)}ms`)
    }

    if (result.status === "err") throw new AuraIpError(result.
error)
    return result.data
  }
}

```

3. Command implementation pattern — [crates/aura-app/src/ipc/commands.rs](https://crates.io/crates/aura-app/src/ipc/commands.rs)

```

use tauri::State;

use crate::ipc::contract::*;
use crate::state::AppState;

/// Every command follows this shape exactly:
/// 1. no business logic here, only translation
/// 2. never blocks the UI thread: real work goes to a blocking task
/// 3. errors become IpcError through one conversion, never ad hoc strings
#[tauri::command]
pub async fn start_import(
    state: State<'_, AppState>,
    request: ImportRequest,
) -> IpcResult<String> {
    let outcome = tauri::async_runtime::spawn_blocking({
        let state = state.inner().clone();
        move || state.begin_import(request)
    }).await;

    match outcome {
        Ok(Ok(run_id)) => IpcResult::Ok { data: run_id },
        Ok(Err(e)) => IpcResult::Err { error: e.into() },
        // A JoinError means the blocking task panicked. `panic = "abort"` in
        // release means this is unreachable in production, but in debug builds
        // it must still not hang the UI waiting for a reply that never comes.
        Err(_) => IpcResult::Err { error: IpcError::internal(
            "import task panicked" ) },
    }
}

```

```
impl From<aura_core::AuraError> for IpcError {
    /// The single conversion point. This is where the privacy guarantee is
    /// enforced: `detail` and `context` never cross the boundary, only
    /// `user_message`, which is asserted jargon-free by a test in aura-core.
    fn from(e: aura_core::AuraError) -> Self {
        Self {
            code: e.code.0.to_string(),
            user_message: e.user_message.clone(),
            severity: e.severity.as_str().to_string(),
            recovery: e.recovery.as_str().to_string(),
            runbook_url: e.code.runbook_url(),
            retryable: e.is_retryable(),
        }
    }
}
```

4. Honest progress — `crates/aura-app/src/progress.rs`

```
use std::sync::{Arc, Mutex};

use aura_core::clock::Clock;
use tauri::{AppHandle, Emitter};

use crate::ipc::contract::Progress;

/// Emits at most 10 events per second, and only when something changed.
///
/// The naive implementation emits per file. At 4,000 files that is 4,000 IPC
/// messages and 4,000 React renders, which makes the UI slow
```

```

er than the import
/// and is the classic reason import screens feel broken.
pub struct ProgressEmitter {
    app: AppHandle,
    clock: Arc<dyn Clock>,
    inner: Mutex<Inner>,
}

struct Inner {
    last_emit_ms: i64,
    last_done: u64,
    started_ms: i64,
    /// Sliding window of (elapsed_ms, done) samples for a rate that reflects
    /// now, not the average since the beginning.
    window: std::collections::VecDeque<i64, u64>,
    monotonic_done: u64,
}

impl ProgressEmitter {
    /// Push an update. Returns immediately; throttling is internal.
    pub fn update(&self, mut p: Progress) {
        let mut inner = match self.inner.lock() {
            Ok(g) => g,
            Err(poisoned) => poisoned.into_inner(), // never lose progress to a poisoned lock
        };
        let now = self.clock.monotonic_ms();

        // INVARIANT: progress never goes backwards. If a retry recounts, we
        // hold the maximum. A bar that jumps back destroys trust instantly.
        inner.monotonic_done = inner.monotonic_done.max(p.done);
    }
}

```

```

        p.done = inner.monotonic_done;

        let is_final = p.total.is_some_and(|t| p.done >= t);
        if !is_final && now - inner.last_emit_ms < 100 && p.d
one == inner.last_done {
            return;
        }

        inner.window.push_back((now, p.done));
        while inner.window.front().is_some_and(|(t, _)| now -
t > 10_000) {
            inner.window.pop_front();
        }

        p.eta_seconds = Self::honest_eta(&inner, now, &p);
        inner.last_emit_ms = now;
        inner.last_done = p.done;
        drop(inner);

        // A failed emit must never fail an import. Log and c
ontinue.
        if let Err(e) = self.app.emit("aura://progress", &p)
{
            tracing::debug!(target: "ui", error = %e, "progre
ss emit failed");
        }
    }

    /// Return an ETA only when it is defensible.
    ///
    /// Three gates, all required:
    /// 1. at least 5 % done, so the sample is not one luck
y small file
    /// 2. total is known at all
    /// 3. the recent rate is within 2x of the overall rat
e, i.e. the run is

```

```

    ///      behaving steadily rather than about to hit 400 h
uge RAWs
    ///
    /// If any gate fails we return None and the UI shows not
hing. "No estimate"
    /// is honest; "3 minutes remaining" that becomes 20 is n
ot.
    fn honest_eta(inner: &Inner, now: i64, p: &Progress) -> O
ption<u64> {
        let total = p.total?;
        if total == 0 || p.done == 0 { return None; }
        if (p.done as f64) / (total as f64) < 0.05 { return N
one; }

        let (t0, d0) = *inner.window.front()?;
        let window_ms = now - t0;
        let window_done = p.done.saturating_sub(d0);
        if window_ms < 1_000 || window_done == 0 { return Non
e; }

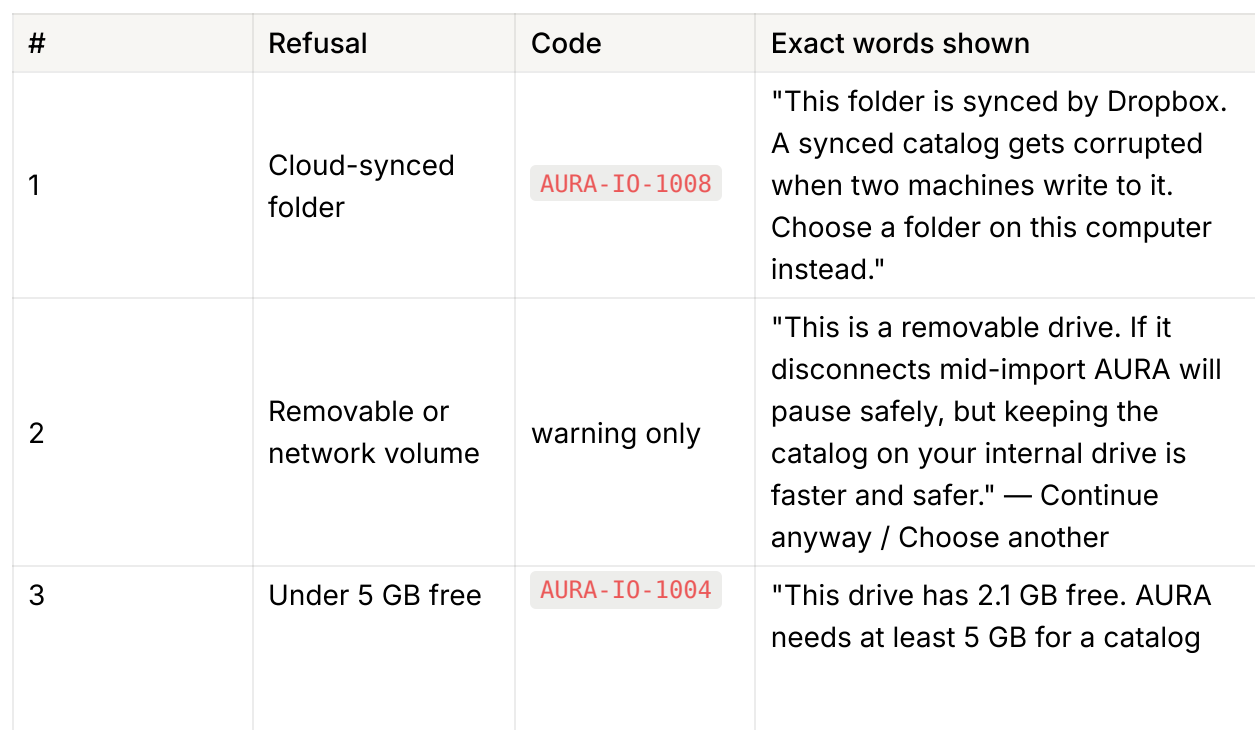
        let recent_rate = (window_done as f64) / (window_ms a
s f64);
        let overall_ms = now - inner.started_ms;
        if overall_ms <= 0 { return None; }
        let overall_rate = (p.done as f64) / (overall_ms as f
64);
        if overall_rate <= 0.0 { return None; }

        let ratio = recent_rate / overall_rate;
        if ratio < 0.5 || ratio > 2.0 { return None; } // u
nstable, refuse to guess

        let remaining = total.saturating_sub(p.done) as f64;
        Some(((remaining / recent_rate) / 1000.0).ceil() as u
64)

```

5. First-run flow and the four refusals



#	Refusal	Code	Exact words shown
			and previews. Free up space or pick another drive."
4	Not writable	AURA-IO-1002	"AURA cannot write to this folder. Pick a folder you own, such as your Pictures folder."

No "Skip" button on the consent screen. Every toggle starts off and staying off is a complete, valid, fully functional configuration — that is what makes the offline promise real rather than decorative.

6. Grid stub — 4,000 cells at 60 fps

```
// ui/src/screens/GridScreen.tsx
import { useVirtualizer } from "@tanstack/react-virtual"
import { memo, useRef } from "react"
import type { GridItem } from "../ipc/types"

const CELL = 180
const GAP = 8

/**
 * Windowed grid. Only visible rows exist in the DOM.
 *
 * Phase 01 renders placeholders with capture time, because Tier-1 thumbnails
 * arrive in PHASE-02. The point of this screen now is to prove the data path
 * and the frame budget with 4,000 real rows, so that PHASE-02 only has to add
 * pixels rather than rebuild scrolling.
 */
export const GridScreen = memo(function GridScreen({ items }: {
  items: GridItem[]
}) {
  const parentRef = useRef<HTMLDivElement>(null)
  const perRow = 6
```

```

const rows = Math.ceil(items.length / perRow)

const virtualizer = useVirtualizer({
  count: rows,
  getScrollElement: () => parentRef.current,
  estimateSize: () => CELL + GAP,
  overscan: 4,    // four rows of headroom: enough to hide fetch latency,
                  // small enough that a fast flick does not build 200 nodes
})

return (
  <div ref={parentRef} className="grid-scroll" style={{ height: "100%", overflow: "auto" }}>
    <div style={{ height: virtualizer.getTotalSize(), position: "relative" }}>
      {virtualizer.getVirtualItems().map((row) => (
        <div key={row.key} className="grid-row"
          style={{ position: "absolute", top: row.start, height: CELL, display: "flex", gap: GAP }}>
          {items.slice(row.index * perRow, row.index * perRow + perRow).map((item) => (
            <Cell key={item.photoId} item={item} />
          ))}
        </div>
      ))}
    </div>
  </div>
)
})

/** memo + a stable key means a progress tick does not re-render 4,000 cells. */
const Cell = memo(function Cell({ item }: { item: GridItem }) {
  {

```

```

return (
  <div className="cell" style={{ width: CELL, height: CELL
}}>
    {item.thumb
      ? <img src={item.thumb} width={CELL} height={CELL} lo
ading="lazy" alt="" />
      : <div className="cell-placeholder">{item.captureTim
e?.slice(11, 16) ?? "-"}</div>}
    </div>
  )
})

```

Progress hook that does not melt the grid

```

// ui/src/hooks/useProgress.ts
import { listen } from "@tauri-apps/api/event"
import { useEffect, useRef, useState } from "react"
import type { Progress } from "../ipc/types"

/**
 * Rust throttles to 10 Hz; we additionally coalesce to one R
eact render per
 * animation frame. Two layers of throttling because the two
systems have
 * different natural rates and neither should have to know ab
out the other.
 */
export function useProgress(): Progress | null {
  const [progress, setProgress] = useState<Progress | null>(n
ull)
  const pending = useRef<Progress | null>(null)
  const frame = useRef<number | null>(null)

  useEffect(() => {
    const unlisten = listen<Progress>("aura://progress", (eve

```

```

nt) => {
  pending.current = event.payload
  if (frame.current !== null) return
  frame.current = requestAnimationFrame(() => {
    frame.current = null
    if (pending.current) setProgress(pending.current)
  })
})
return () => {
  if (frame.current !== null) cancelAnimationFrame(frame.current)
  void unlisten.then((f) => f())
}
}, [])

return progress
}

```

7. Tauri security configuration — `crates/aura-app/tauri.conf.json`

```

{
  "productName": "AURA",
  "identifier": "app.aura.desktop",
  "build": { "frontendDist": "../..ui/dist", "devUrl": "http://localhost:5173" },
  "app": {
    "windows": [{
      "title": "AURA",
      "width": 1440, "height": 900,
      "minWidth": 1100, "minHeight": 700,
      "dragDropEnabled": true
    }],
    "security": {
      "csp": "default-src 'self'; img-src 'self' asset: http

```

```

p://asset.localhost blob: data;; style-src 'self' 'unsafe-inl
ine'; script-src 'self'; connect-src 'self' ipc: http://ipc.l
ocalhost; font-src 'self'; object-src 'none'; frame-src 'non
e'",
    "assetProtocol": { "enable": true, "scope": ["$APPCACH
E/**"] },
    "freezePrototype": true
  }
},
  "plugins": {}
}

```

Setting	Why it is this way
No <code>connect-src https:</code>	Phase 01 makes zero network calls. The CSP proves it rather than asserting it — a stray <code>fetch</code> fails at runtime and in tests
<code>asset</code> scope limited to <code>\$APPCACHE</code>	The webview can display generated thumbnails and nothing else. It can never read a client's RAW folder
<code>object-src 'none'</code> , <code>frame-src 'none'</code>	No plugins, no iframes; removes a whole class of webview escapes
<code>freezePrototype: true</code>	Prototype pollution from any future dependency cannot rewrite our IPC wrapper
<code>dragDropEnabled: true</code>	Dragging a card folder onto the window is the fastest possible import; it goes through the same validated path as the picker

8. Tests for T13–T15

```

// crates/aura-app/tests/ipc.rs

#[test]
fn every_command_in_commands_v1_is_registered() {
    // Parse the generate_handler! invocation and compare wit
h COMMANDS_V1.
    // A command added to one and not the other is a silent 4
04 at runtime.

```

```

    let registered = parse_registered_commands(include_str!(
    ("../src/lib.rs")));
    let mut declared: Vec<&str> = COMMANDS_V1.to_vec();
    declared.sort_unstable();
    assert_eq!(registered, declared);
}

#[test]
fn ipc_error_never_contains_a_path_or_jargon() {
    for err in every_error_in_registry() {
        let ipc: IpcError = err.into();
        for banned in ["/", "\\\\" , "sqlite", "rusqlite", "pa
nic", "unwrap",
                        "Err(", "None", "blake3", "thread"] {
            assert!(!ipc.user_message.contains(banned),
                    "{} leaks {banned:?}: {}", ipc.code, ipc.user
_message);
        }
        assert!(ipc.runbook_url.starts_with("aura.app/e/"));
    }
}

#[test]
fn rust_and_typescript_contracts_agree() {
    // Regenerate types from the Rust contract and diff again
    st the committed
    // ui/src/ipc/types.ts. This is the test that keeps the m
    irror honest.
    let generated = aura_app::xtask::generate_ts();
    let committed = include_str!("../../ui/src/ipc/types.t
s");
    assert_eq!(generated.trim(), committed.trim(),
               "run `cargo xtask ipc-types` and commit the result");
}

#[test]

```

```

fn progress_is_monotonic_even_when_a_retry_recounts() {
    let emitter = test_emitter();
    emitter.update(progress_with(done = 500));
    emitter.update(progress_with(done = 300)); // a retry r
    ecounted downward
    assert_eq!(last_emitted().done, 500);
}

#[test]
fn eta_is_none_until_five_percent_and_none_when_rate_is_unstable() {
    let emitter = test_emitter();
    emitter.update(progress_with(done = 10, total = Some(4000)));
    assert!(last_emitted().eta_seconds.is_none(), "too early to estimate");

    feed_steady_rate(&emitter, 0.30);
    assert!(last_emitted().eta_seconds.is_some(), "steady run should estimate");

    feed_sudden_stall(&emitter);
    assert!(last_emitted().eta_seconds.is_none(), "unstable rate must not estimate");
}

#[test]
fn progress_emits_at_most_ten_times_per_second() {
    let emitter = test_emitter(); // FixedClock
    for i in 0..1000 { emitter.update(progress_with(done = i)); clock().advance_ms(1); }
    assert!(emitted_count() <= 11, "emitted {} times in 1 s", emitted_count());
}

#[test]

```

```

fn progress_current_item_is_a_file_name_never_a_path() {
    let p = build_progress_for("/Users/x/Weddings/Smith/DAY1/
IMG_0421.CR2");
    assert_eq!(p.current_item.as_deref(), Some("IMG_0421.CR
2"));
}

#[test]
fn first_run_refuses_a_cloud_synced_catalog_location() {
    let err = choose_catalog_location(dropbox_path()).err().e
xpect("must refuse");
    assert_eq!(err.code, "AURA-IO-1008");
    assert!(err.user_message.contains("synced"));
}

#[test]
fn first_run_refuses_when_free_space_is_below_five_gb() { /*
AURA-IO-1004 */ }

#[test]
fn new_project_consent_is_all_false() {
    let p = create_project("Smith Wedding", None).expect("cre
ate");
    let c = p.consent;
    assert!(!c.cloud_metadata && !c.cloud_derived_imagery &&
!c.cloud_full_imagery
        && !c.telemetry && !c.improve_models);
    assert!(c.granted_at.is_none());
}

#[test]
fn cancel_is_acknowledged_within_500ms() {
    let run = start_import_of(4000);
    let t0 = std::time::Instant::now();
    cancel_run(&run).expect("cancel");
    wait_for_state(&run, "cancelled");
}

```

```

    assert!(t0.elapsed().as_millis() < 500, "cancel took {} ms", t0.elapsed().as_millis());
}

#[test]
fn no_network_call_is_made_during_first_run_or_import() {
    // Bind a listener on the loopback and assert zero connections; additionally
    // assert the binary contains no reference to a remote host in phase 01.
    let guard = NetworkGuard::deny_all();
    complete_first_run_and_import_400();
    assert_eq!(guard.attempts(), 0, "phase 01 must make no network call");
}

```

```

// ui/src/ipc/client.test.ts
import { describe, expect, it, vi } from "vitest"
import { AuraIpError, call } from "./client"

describe("ipc client", () => {
    it("turns an err result into an AuraIpError carrying the code", async () => {
        mockInvoke({ status: "err", error: { code: "AURA-IO-1003", retryable: true, /* ... */ } })
        await expect(call("start_import", {})).rejects.toBeInstanceOf(AuraIpError)
        await expect(call("start_import", {})).rejects.toMatchObject({ detail: { code: "AURA-IO-1003" } })
    })

    it("turns a thrown invoke into a fatal internal error, not a silent undefined", async () => {
        mockInvokeThrows(new Error("command not found"))
        await expect(call("nope", {})).rejects.toMatchObject({ de

```

```

tail: { code: "AURA-IPC-9001" } })
    })
})

describe("progress rendering", () => {
    it("renders an indeterminate bar with a count when total is
null", () => { /* ... */ })
    it("renders no ETA text at all when etaSeconds is null", ()
=> { /* ... */ })
    it("never renders a percentage above 100", () => { /* ...
*/ })
})

```

9. Acceptance criteria for T13–T15

- ☐ App launches on Windows, macOS and Linux; cold launch under 2.5 s on the reference machine, number pasted per OS
- ☐ All 14 commands in `COMMANDS_V1` are registered; the parity test passes
- ☐ `cargo xtask ipc-types` produces byte-identical output to the committed `ui/src/ipc/types.ts`
- ☐ Every registry error converts to an `IpError` with a code, a runbook URL, and a `user_message` containing no path, no jargon and no Rust type names
- ☐ Progress is monotonic under a recounting retry
- ☐ ETA is absent below 5 % and absent when the rate is unstable; present and stable during a steady run
- ☐ Emission rate at most 10 Hz measured over a simulated second
- ☐ `current_item` is a bare file name in every emitted event
- ☐ First run refuses a cloud-synced location, an unwritable folder and a drive with under 5 GB free, each with its exact code and wording
- ☐ A removable volume warns but can be accepted deliberately
- ☐ A newly created project has every consent flag false and `granted_at` null

- ☐ Cancel acknowledged in under 500 ms; measured value pasted
- ☐ 4,000-cell grid scrolls with no frame over 16.7 ms in a 10-second scroll trace; trace summary pasted
- ☐ Zero network connections during first run and import, proven by the network guard
- ☐ CSP contains no `https:` in `connect-src`; asset scope is `$APPCACHE` only
- ☐ `npm run lint`, `tsc --noEmit`, and `vitest run` all clean; no `any` in `ui/src/ipc`

10. Claude Code prompt for T13–T15

Act as `SFE` (Senior Frontend) with `MBE` (Backend) on the Rust side, and review under the `SEC` hat. Implement T13, T14 and T15 of PHASE-01 from this page.

Copy `crates/aura-app/src/ipc/contract.rs` and `ui/src/ipc/types.ts` **verbatim**, then write the `cargo xtask ipc-types` generator that reproduces the TypeScript file from the Rust file and wire it into CI so the two can never drift. Implement `commands.rs` for all 14 commands, `progress.rs` exactly as specified including the three ETA gates, the first-run flow with all four refusals and their exact wording, the typed `call` wrapper, `useProgress`, and the virtualised 4,000-cell grid.

Hard constraints: no command may block the UI thread; no error may reach the UI as a bare string; `detail` and `context` from `AuraError` must never cross the IPC boundary; progress must be monotonic and throttled to 10 Hz; an ETA that fails any gate is `None`, never a placeholder string; no network call anywhere in phase 01; no `any` in `ui/src/ipc`.

Evidence I expect pasted back: `cargo test -p aura-app` and `vitest run` in full; the command-parity and types-parity test output; cold launch times for all three platforms; the cancel-latency measurement; a 10-second scroll performance trace summary for the 4,000-cell grid; and the network-guard assertion output. Stop after T15.